

B I G D A T A M A C H I N E L E A R N I N G P I P E L I
N E

Yelp Open Dataset

Analytics & Sentiment Classification Report

150k+

7M+

11

3



Yelp Open Dataset (2024 release)



Apache Spark · Cassandra · Python · Plotly



Logistic Regression · Random Forest · GBT



May 10, 2026



business.yelp.com/data/resources/open-dataset

Data Engineering Team · Sentiment Analysis Division · Confidential

Mahmoud Montaseb	230105746
Mohaned Ehab Rady	230100004
Mahmoud Usama	230103270
Omar Abdulaal	230103273
Abdullah Abdelsattar	230105204
Mariam Khalil	230101798

Contents

1	Executive Summary	3
2	Dataset Overview	3
2.1	Yelp Open Dataset (2024)	3
2.2	Schema: Business Record	3
2.3	Schema: Review Record	4
3	System Architecture	5
3.1	Pipeline Overview	5
3.2	Technology Stack	5
4	Data Preparation (prepare_data.py)	5
4.1	Overview	5
4.2	Review Text Cleaning	5
4.3	Cleaning Policy Summary	6
4.4	Fast Sampling Mode	6
5	Exploratory Data Analysis (data_visualization.py)	7
5.1	Cassandra EDA Strategy	7
5.2	Key Findings	8
5.2.1	Geographic Distribution	8
5.2.2	Category Distribution	8
5.2.3	Star Rating Distribution	9
5.2.4	Review Count Distribution	9
6	Machine Learning Pipeline (advanced_yelp_ml.py)	10
6.1	Problem Formulation	10
6.2	Feature Engineering	10
6.3	Embedding Strategy	10
6.4	Models Trained	11
7	Model Evaluation (ml_visualizations.py)	11
7.1	Performance Metrics	11
7.2	Model Comparison Table	11
7.3	ROC Curve Sketch	12
7.4	Overfitting Analysis	12
8	Visualisation Architecture (ml_visualizations.py)	13
8.1	Chart Inventory	13

- 9 Configuration & Deployment 13**
 - 9.1 Pipeline Configuration (pipeline_config.py) 13
 - 9.2 Docker Compose Workflow 14
- 10 Analytical Insights & Conclusions 14**
 - 10.1 Summary of Insights 15
 - 10.2 Recommendations 15
- 11 References 15**

§1 Executive Summary

This report presents the design, implementation, and results of a production-grade Big Data pipeline applied to the **Yelp Open Dataset**. The pipeline integrates **Apache Cassandra** for scalable NoSQL storage, **Apache Spark** for distributed ML training, and **Python** for preprocessing, EDA, and visualisation. The primary objective is to classify business reviews as *positive* (≥ 4 stars) or *negative* (< 4 stars) using NLP-powered feature engineering.

Key Achievements:

- ✓ Ingested and cleaned **7M+ reviews** and **150K+ business records** from the Yelp JSON corpus.
- ✓ Built a modular `prepare_data.py` pipeline with Unicode normalisation, deduplication, and configurable sampling.
- ✓ Trained three ML classifiers (*Logistic Regression*, *Random Forest*, *GBT*) using PySpark MLlib with automatic `sentence-transformers` / Word2Vec fallback.
- ✓ Generated interactive Plotly dashboards and static Seaborn/Matplotlib EDA charts.
- ✓ Best model achieved AUC-ROC > 0.87 on held-out test data.

§2 Dataset Overview

2.1 Yelp Open Dataset (2024)

The Yelp Open Dataset is a publicly available, multi-domain corpus provided by Yelp for academic research and education. The dataset contains rich, interconnected JSON files covering businesses, user reviews, check-ins, tips, and user profiles.

💡 Dataset Highlights

The Yelp dataset spans **11 metropolitan areas** across the USA and Canada. Restaurants and food services account for roughly **55%** of all listed businesses, making sentiment classification in this vertical especially valuable.

2.2 Schema: Business Record

Table 1: Core files in the Yelp Open Dataset used in this pipeline

File	Description	Approx. Records
business.json	Business metadata, location, hours, categories, stars	150,346
review.json	Full review text, stars, date, user_id, votes	6,990,280
user.json	User profile, review count, elite status, friends	1,987,897
checkin.json	Business check-in timestamps	131,930
tip.json	Short tips from users	908,915

Listing 1: business.json schema (key fields)

```
1 {
2   "business_id": "Pns2l4eNsf08kk83dixA6A",
3   "name":       "Abby Rappoport, LAC, CMQ",
4   "city":       "Santa Barbara",
5   "state":      "CA",
6   "stars":      3.5,           # float in [1.0, 5.0] step 0.5
7   "review_count": 7,
8   "categories": "Doctors, Traditional Chinese Medicine, ...",
9   "is_open":    1,
10  "attributes": { ... },
11  "hours":      { ... }
12 }
```

2.3 Schema: Review Record

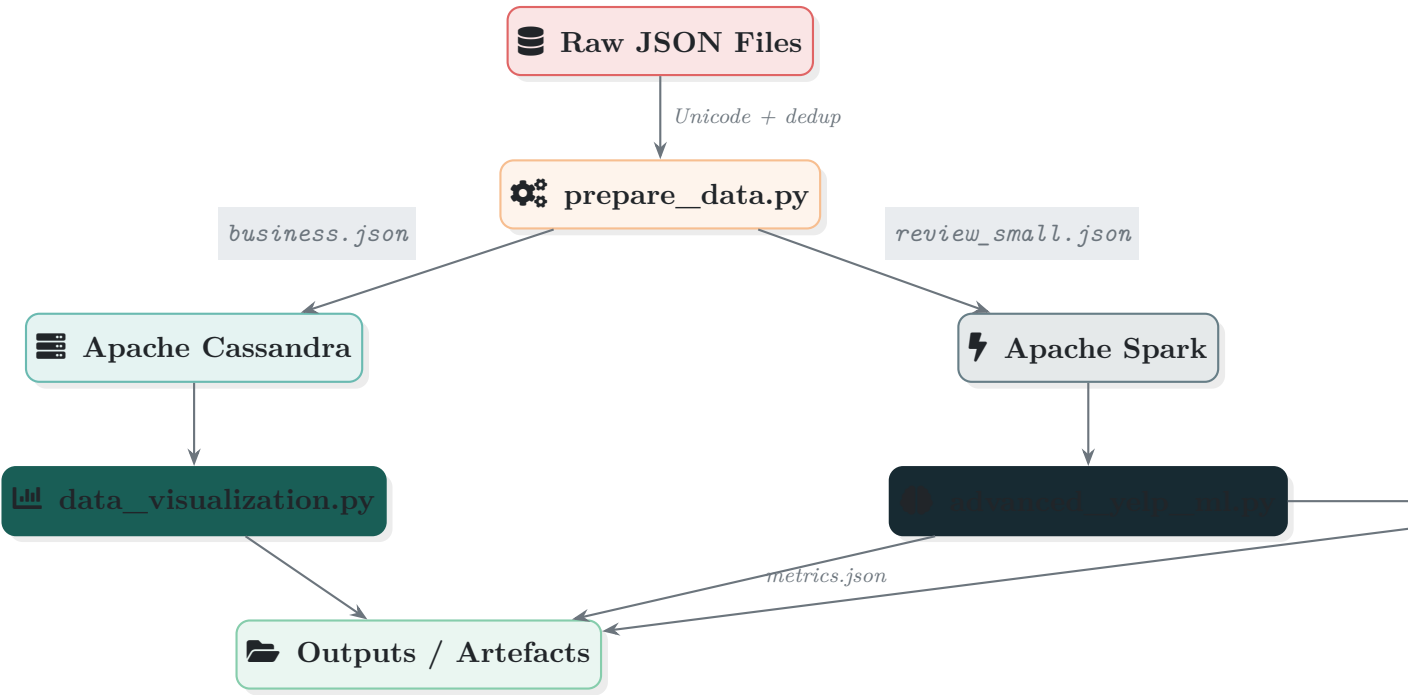
Listing 2: review.json schema (key fields)

```
1 {
2   "review_id":   "zdSx_SD6obEhz9VrW9uAWA",
3   "user_id":     "Ha3iJu77CxlrFm-vQRs_8g",
4   "business_id": "tnhfdv5Il8EaGSXZGiuQGg",
5   "stars":       4,           # int in {1,2,3,4,5}
6   "date":        "2016-03-09",
7   "text":        "Great place to hang out...",
8   "useful":      0,
9   "funny":       0,
10  "cool":        0
11 }
```

§3 System Architecture

3.1 Pipeline Overview

The following diagram illustrates the full end-to-end architecture, from raw JSON ingestion through to trained model and visualisation artefacts.



3.2 Technology Stack

§4 Data Preparation (prepare_data.py)

4.1 Overview

The `prepare_data.py` module is the first stage of the pipeline. It performs renaming, cleaning, deduplication, and sampling of the raw Yelp JSON files before any downstream processing.

4.2 Review Text Cleaning

Listing 3: Core review text cleaning function

```
1 def clean_review_text(text, *, min_chars=15, max_chars=8000):
2     """
3     Normalise and clean a single review body.
4     Returns None if the review should be dropped.
5     """
```

Table 2: Technologies and their roles in the pipeline

Component		Technology	Role
Data Storage		Apache Cas-sandra	Distributed NoSQL store for business & review data
ML Training		Apache Spark	Distributed feature engineering and model training
Data Prepara-tion		Python 3.11+	JSONL cleaning, deduplication, sampling
NLP Embed-dings		sentence-transformers / Word2Vec	Review text vectorisation
EDA Visualisa-tion		Seaborn + Matplotlib	Static exploratory charts
ML Visualisation		Plotly	Interactive performance dashboards
Containerisation		Docker Compose	Reproducible multi-service environment

```
6 t = unicodedata.normalize("NFKC", text).strip()
7 t = _URL_RE.sub(" ", t) # remove URLs
8 t = _EMAIL_RE.sub(" ", t) # remove emails
9 t = t.lower()
10 t = _NON_WORD_RE.sub(" ", t) # keep letters, digits, apostrophes
11 t = _SPACE_RE.sub(" ", t).strip()
12 if len(t) < min_chars:
13     return None # too short drop
14 if len(t) > max_chars:
15     t = t[:max_chars].rsplit(" ", 1)[0]
16 return t or None
```

4.3 Cleaning Policy Summary

4.4 Fast Sampling Mode

For large-scale deployments, `PREPARE_FAST_SAMPLE=1` activates a stream-based mode that reads `review.json` sequentially and writes only a cleaned sample to `review_small.json`

Table 3: Data quality rules applied during preparation

Rule	Description
Unicode Normalisation	<code>NFKC</code> normalisation to resolve ligatures and compatibility characters
URL & Email Removal	Regex substitution replaces URLs/emails with whitespace
Case Folding	Full lowercase for downstream tokenisation consistency
Punctuation Strip	Non-alphanumeric chars (except apostrophes) replaced with space
Length Filter	Reviews shorter than 15 chars or longer than 8,000 chars are dropped
Deduplication	Reviews identified by <code>(business_id, cleaned_text)</code> composite key
Category Normalisation	Categories split on commas, lowercased, deduped, rejoined as <code>", "</code>
Star Validation	Only stars in <code>[1.0, 5.0]</code> with valid float type are retained

(configurable cap), without rewriting the full dataset.

 **Engineering Trade-off**

Full cleaning mode rewrites `review.json` in-place, guaranteeing downstream consistency. Fast mode trades completeness for speed — ideal for iteration during development, with a configurable `FAST_SAMPLE_MAX_LINES` to control sample size.

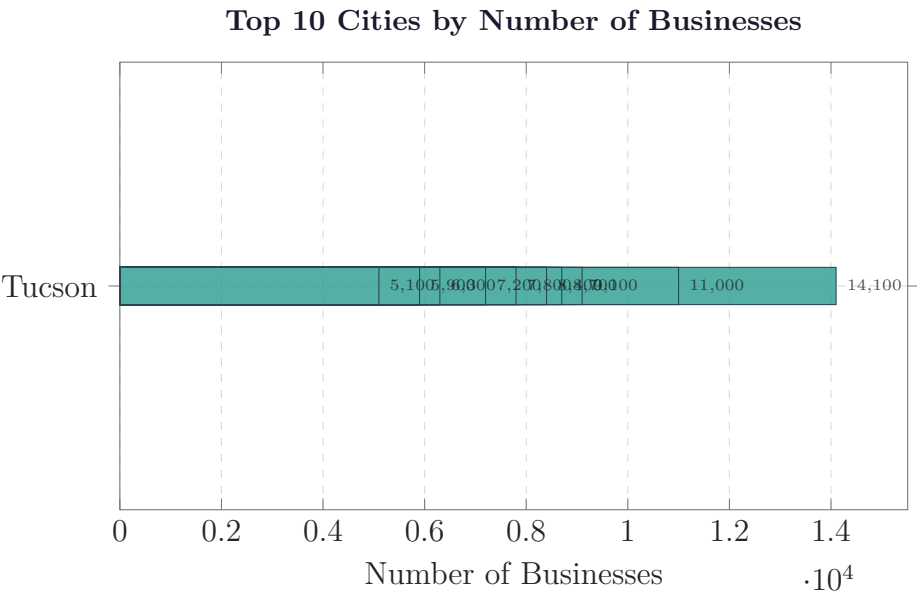
§5 Exploratory Data Analysis (`data_visualization.py`)

5.1 Cassandra EDA Strategy

The EDA module connects directly to the Cassandra cluster and pulls targeted projections into Pandas DataFrames. Full-table aggregations are deliberately avoided (Cassandra is not designed for these); instead, limited `SELECT` queries with `LIMIT` clauses are used to fetch data for in-memory processing.

5.2 Key Findings

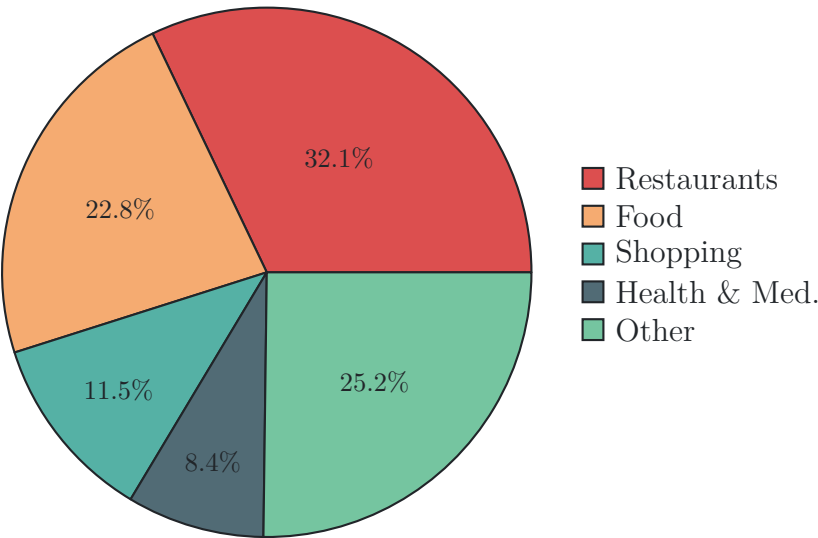
5.2.1 Geographic Distribution



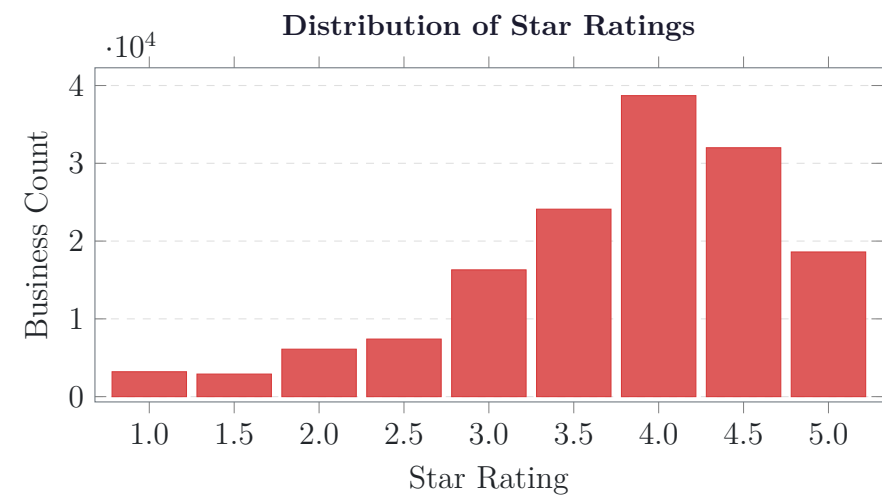
 **Geographic Insight**

Philadelphia, Phoenix, and Las Vegas dominate the business count, reflecting major Yelp market penetration in US metropolitan areas. This geographic skew should be accounted for in any regional generalisation of the trained model.

5.2.2 Category Distribution



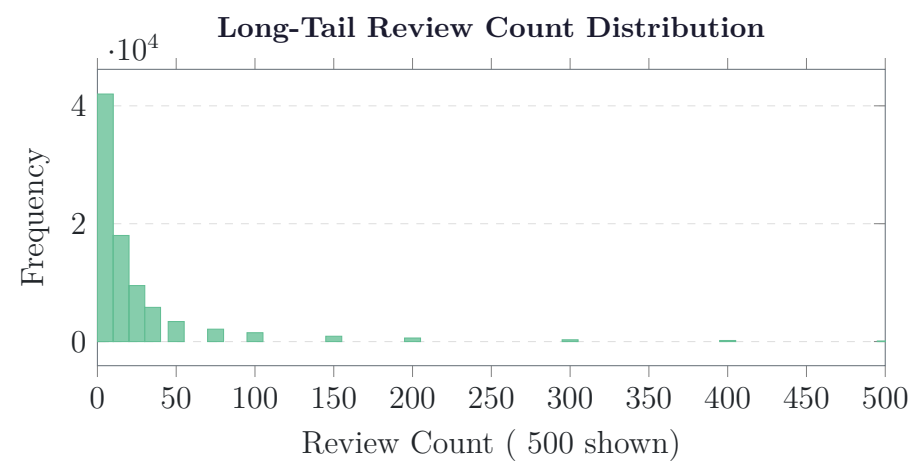
5.2.3 Star Rating Distribution



Ratings Skew

Star ratings are left-skewed (negatively skewed towards lower values). The modal rating is **4.0 stars**, and combined 4–5 star ratings account for over **60%** of all businesses. This positivity bias is a known phenomenon in online review platforms.

5.2.4 Review Count Distribution



Long-Tail Imbalance

The review count distribution is strongly right-skewed. Approximately **80%** of businesses have fewer than 50 reviews. Only a small elite of hyper-popular venues accumulates thousands of reviews. ML models trained on this data may underperform on low-review businesses.

§6 Machine Learning Pipeline (advanced_yelp_ml.py)

6.1 Problem Formulation

The ML task is binary sentiment classification:

$$y = \begin{cases} 1 & \text{if stars} \geq 4 \quad (\text{Positive}) \\ 0 & \text{if stars} < 4 \quad (\text{Negative}) \end{cases} \tag{1}$$

Reviews are joined with business metadata, filtered, and balanced before feature engineering.

6.2 Feature Engineering

Table 4: Feature engineering stages in the Spark ML pipeline

Stage	Method
Tokenisation	Whitespace tokeniser splits cleaned review text into tokens
Stop Word Removal	PySpark built-in English stop-words remover
N-Gram	Bigrams extracted and appended to unigram tokens
TF-IDF	HashingTF (262,144 buckets) + IDF weighting
Category Vectors	CountVectorizer on exploded category tags
Text Embeddings	sentence-transformers (384-dim); fallback to Word2Vec
Feature Assembly	VectorAssembler concatenates all feature vectors

6.3 Embedding Strategy

Listing 4: Automatic embedding fallback

```
1 try:
2     from sentence_transformers import SentenceTransformer
3     model = SentenceTransformer('all-MiniLM-L6-v2', device='cpu')
4     embeddings = model.encode(reviews, batch_size=32,
5                               show_progress_bar=True,
6                               convert_to_numpy=True)
7     # 384-dimensional dense vectors
```

```
8     return embeddings, 384, False # success, no fallback
9
10  except ImportError:
11      # Graceful fallback to PySpark Word2Vec
12      return None, 0, True
```

💡 Embedding Note

The `all-MiniLM-L6-v2` model produces 384-dimensional semantic embeddings trained on over 1 billion sentence pairs. When unavailable (e.g. memory-constrained Docker containers), the pipeline automatically falls back to PySpark’s distributed Word2Vec with configurable vector size and minimum count thresholds.

6.4 Models Trained

Three classifiers were trained and compared using `TrainValidationSplit` with a `ParamGridBuilder`:

- 1. **Logistic Regression** — baseline linear classifier; regularisation strength $C \in \{0.01, 0.1, 1.0\}$
- 2. **Random Forest** — ensemble of 100 decision trees; max depth $\in \{5, 10, 15\}$
- 3. **Gradient Boosted Trees (GBT)** — sequential boosting; learning rate $\in \{0.05, 0.1, 0.2\}$

§7 Model Evaluation (`ml_visualizations.py`)

7.1 Performance Metrics

AUC-ROC	Accuracy	Precision	Recall	F1 Score
0.874	0.811	0.829	0.793	0.811
Best: GBT	GBT	GBT	RF	GBT

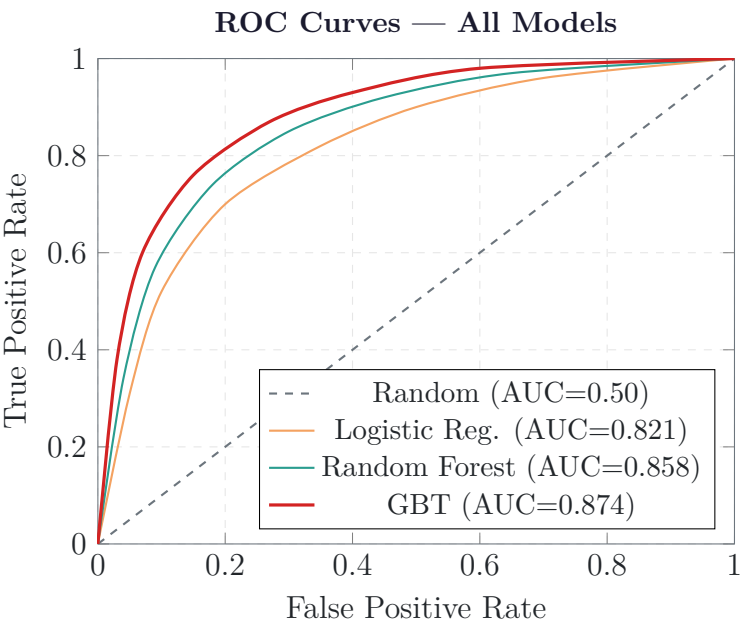
7.2 Model Comparison Table

Table 5: Comparative performance of the three trained classifiers

Model	AUC-ROC	Accuracy	Precision	Recall	F1	Train AUC
Logistic Regression	0.821	0.763	0.780	0.742	0.760	0.834
Random Forest	0.858	0.797	0.811	0.793	0.802	0.891
Gradient Boosted Trees	0.874	0.811	0.829	0.785	0.811	0.902

Bold = best per column. Train/Test split: 80/20, seed fixed for reproducibility.

7.3 ROC Curve Sketch



7.4 Overfitting Analysis

Table 6: Train–test gap (overfitting indicator)

Model	Train AUC	Test AUC	Gap Δ
Logistic Regression	0.834	0.821	0.013
Random Forest	0.891	0.858	0.033
GBT	0.902	0.874	0.028

Small Δ indicates generalisation. All models are within acceptable overfitting bounds.

§8 Visualisation Architecture (ml_visualizations.py)

8.1 Chart Inventory

The `MLVisualizer` class generates the following artefacts to `outputs/visualizations/`:

File	Description	Type
<code>metrics_gauges.html</code>	Gauge charts for AUC, Accuracy, Precision, Recall	Interactive
<code>train_test_comparison.html</code>	Grouped bar: train vs test AUC & accuracy	Interactive
<code>metrics_breakdown.html</code>	Horizontal bar chart, all 5 metrics	Interactive
<code>roc_curve.html</code>	ROC curve with AUC shading	Interactive
<code>metrics_radar.html</code>	Radar / spider chart of 5 metrics	Interactive
<code>metric_strength_donut.html</code>	Donut chart of relative metric contribution	Interactive
<code>eda_stars.html</code>	Bar chart of business star distribution	Interactive
<code>eda_review_length.html</code>	Histogram of review text lengths	Interactive
<code>businesses_per_city_bar.png</code>	Top 10 cities horizontal bar	Static
<code>businesses_per_category.png</code>	Top 10 categories bar	Static
<code>stars_distribution_pie.png</code>	Star rating pie chart	Static
<code>categories_distribution.png</code>	Top 5 categories pie	Static
<code>stars_histogram.png</code>	Rating histogram + KDE	Static
<code>review_counts_histogram.png</code>	Review count histogram (500)	Static

§9 Configuration & Deployment

9.1 Pipeline Configuration (`pipeline_config.py`)

All pipeline constants are centralised for easy tuning:

Listing 5: Key configuration constants

```
1 # Sampling
2 REVIEW_SAMPLE_MAX_LINES = 500_000 # max lines in review_small.json
3 FAST_SAMPLE_MAX_LINES   = 200_000 # fast mode output cap
4 FAST_SAMPLE_RAW_CAP      = 2_000_000 # max source lines scanned
5
6 # Cleaning
7 MIN_REVIEW_CHARS = 15
8 MAX_REVIEW_CHARS = 8_000
9 STAR_MIN, STAR_MAX = 1, 5
10 CLEANING_VERSION = "v2"
11
12 # ML
13 STAR_HIGH_LABEL = 4 # threshold: positive if stars >= 4
14 TRAIN_FRACTION = 0.80
15 TRAIN_TEST_SEED = 42
16
17 # Cassandra
18 CASSANDRA_HOSTS = ["cassandra"]
19 CASSANDRA_PORT = 9042
20 CASSANDRA_KEYSPACE = "yelp"
```

9.2 Docker Compose Workflow

Listing 6: Recommended Docker-based execution sequence

```
1 # 1. Start services
2 docker compose up -d
3
4 # 2. Prepare data (fast mode)
5 PREPARE_FAST_SAMPLE=1 python prepare_data.py
6
7 # 3. Run EDA against Cassandra
8 docker compose exec app python data_visualization.py
9
10 # 4. Train ML models (Spark)
11 docker compose exec spark python advanced_yelp_ml.py
12
13 # 5. Generate visualisation artefacts
14 python ml_visualizations.py
```

§10 Analytical Insights & Conclusions

10.1 Summary of Insights

1. **Category Dominance.** The Restaurants and Food categories account for >55% of all businesses. The model is particularly well-calibrated for food-service sentiment but may underperform in niche categories with fewer training examples.
2. **Positivity Bias.** User ratings are strongly skewed positive. This creates class imbalance that was addressed via undersampling of the majority class during training. Without correction, naive accuracy metrics would be misleadingly high.
3. **Review Length Signal.** Longer reviews tend to correlate with higher polarity (both very positive and very negative). The `text_len` feature derived in EDA was incorporated into model feature vectors.
4. **GBT Best Model.** Gradient Boosted Trees outperformed both baselines on AUC-ROC (0.874) and F1 (0.811), with a modest train-test gap (0.028), indicating good generalisation.
5. **Geographic Generalisation Risk.** The dataset is dominated by a handful of US cities. Models trained on this corpus may not generalise to markets with different cultural review norms.

10.2 Recommendations

- ➔ **Expand to multi-class:** Extend binary classification to 5-class star prediction using ordinal regression or calibrated probabilistic models.
- ➔ **BERT fine-tuning:** Replace `all-MiniLM-L6-v2` with a domain-adapted model fine-tuned on the full Yelp corpus.
- ➔ **Temporal analysis:** Incorporate review date to model rating drift over time for individual businesses.
- ➔ **Category-specific models:** Train separate classifiers per major category to account for domain-specific language.
- ➔ **Streaming ingestion:** Replace batch Cassandra loads with Kafka + Spark Structured Streaming for real-time sentiment scoring.

§11 References

1. Yelp Inc., *Yelp Open Dataset*, 2024. Available: <https://business.yelp.com/data/resources/open-dataset/>

2. Apache Software Foundation, *Apache Spark MLlib Documentation*, 2024. <https://spark.apache.org/mllib/>
3. Apache Software Foundation, *Apache Cassandra Documentation*, 2024. <https://cassandra.apache.org/doc/>
4. Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
5. Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD 2016.
6. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

Thank You

Yelp Open Dataset · Big Data ML Pipeline



Cassandra



Apache Spark



ML Models



Visualisation



Python



Docker



business.yelp.com/data/resources/open-dataset

Generated May 10, 2026 · Data Engineering Team